

Quadruple Square: memoria técnica

Índice

Quadruple Square: memoria técnica.....	1
1. Introducción.....	1
2. Objetivos del sistema.....	1
3. Requisitos y entorno.....	2
4. Arquitectura del proyecto.....	2
5. Sistema de autenticación y sesiones.....	3
6. Modelo de datos.....	3
7. Rutas principales.....	3
8. Lógica funcional del juego.....	5
9. Vistas y experiencia de usuario.....	5
10. Middleware y tratamiento de errores.....	5
11. Retos técnicos y decisiones de implementación.....	6
12. Conclusión.....	6

1. Introducción

Quadruple Square es un proyecto backend desarrollado bajo una arquitectura MVC que implementa un juego de cartas estratégico inspirado en un minijuego de Square Enix. La aplicación permite al usuario registrarse, iniciar sesión, gestionar su colección de cartas, seleccionar un mazo de cinco cartas y enfrentarse a un bot en partidas sobre un tablero de 3x3. El objetivo del juego es capturar el mayor número posible de cartas del rival mediante la colocación táctica de las propias cartas y la comparación de sus valores de poder en los cuatro lados. Al finalizar la partida, el jugador ganador obtiene una de las cartas previamente seleccionadas por su oponente, incorporándola a su colección.

El proyecto ha sido desarrollado como un backend con vistas renderizadas en servidor mediante EJS, junto con una API interna que permite actualizar el estado del juego sin necesidad de recargar la interfaz completa. Para la persistencia de datos se utiliza Sequelize como ORM, y la autenticación se gestiona mediante sesiones con *express-session*, lo que permite mantener el estado del usuario durante la navegación y limitar el acceso a la gestión de admin. El sistema se ha estructurado de forma modular separando controladores, servicios, rutas, vistas, modelos y middlewares, con el objetivo de facilitar el mantenimiento, la escalabilidad y la comprensión del código.

2. Objetivos del sistema

El objetivo principal del proyecto es ofrecer una aplicación funcional que permita simular una partida táctica entre un jugador humano y un bot, gestionando todo el ciclo de juego desde el registro hasta el resultado final. Entre los objetivos específicos se encuentran: permitir el registro e inicio de sesión de usuarios, administrar una colección de cartas por jugador, seleccionar un

conjunto de cinco cartas para cada partida, ejecutar partidas contra bots, aplicar automáticamente las reglas de captura por poder de cartas, gestionar el estado de partida y el cambio de turnos, y otorgar una carta del rival al ganador al final de la partida.

Además, el sistema busca servir como ejemplo de aplicación backend estructurada con buenas prácticas de separación de responsabilidades. Por ello, el código se organiza de manera que la lógica de negocio no se mezcle con la presentación, y las operaciones críticas sobre la base de datos se realicen de forma controlada mediante transacciones. Esto resulta especialmente importante en acciones como la selección de cartas, la cancelación de partidas o la fase final de robo, donde la consistencia de los datos debe estar garantizada.

3. Requisitos y entorno

Para ejecutar el proyecto se requiere disponer de *Node.js*, *npm*, *Docker* y *Docker Compose*, además de un archivo **.env** con las variables de entorno necesarias para la conexión a la base de datos y la configuración de la aplicación. Entre las dependencias principales utilizadas por el proyecto se encuentran Express, EJS, pg, Sequelize y dotenv. La aplicación está preparada para arrancar mediante Docker, lo que simplifica su despliegue y estandariza el entorno de ejecución.

El proyecto incluye además un sistema de datos iniciales mediante un archivo de seed. Al iniciarse la aplicación, si no existen usuarios en la base de datos, se cargan automáticamente datos base que permiten probar el sistema desde el primer arranque. Esto incluye un usuario administrador y configuración inicial para comenzar a interactuar con la plataforma sin necesidad de introducir datos manualmente. En la interfaz de administración, se pueden crear y asignar cartas a los bots para configurar un progreso de juego al gusto del usuario.

4. Arquitectura del proyecto

El proyecto sigue una arquitectura MVC clásica. La capa de modelos define las entidades persistentes y sus relaciones; la capa de servicios concentra la lógica de negocio; la capa de controladores actúa como intermediaria entre la petición HTTP y los servicios; y la capa de vistas se encarga de la representación visual mediante plantillas EJS. Esta separación permite mantener el código ordenado y facilita la evolución del proyecto.

La estructura general del proyecto está organizada de la siguiente manera:

src/controllers contiene los controladores de vistas y API;

src/services concentra la lógica principal del juego y la interacción con la base de datos;

src/models agrupa los modelos de Sequelize;

src/routes define las rutas principales y sus subrutas;

src/views contiene las plantillas EJS;

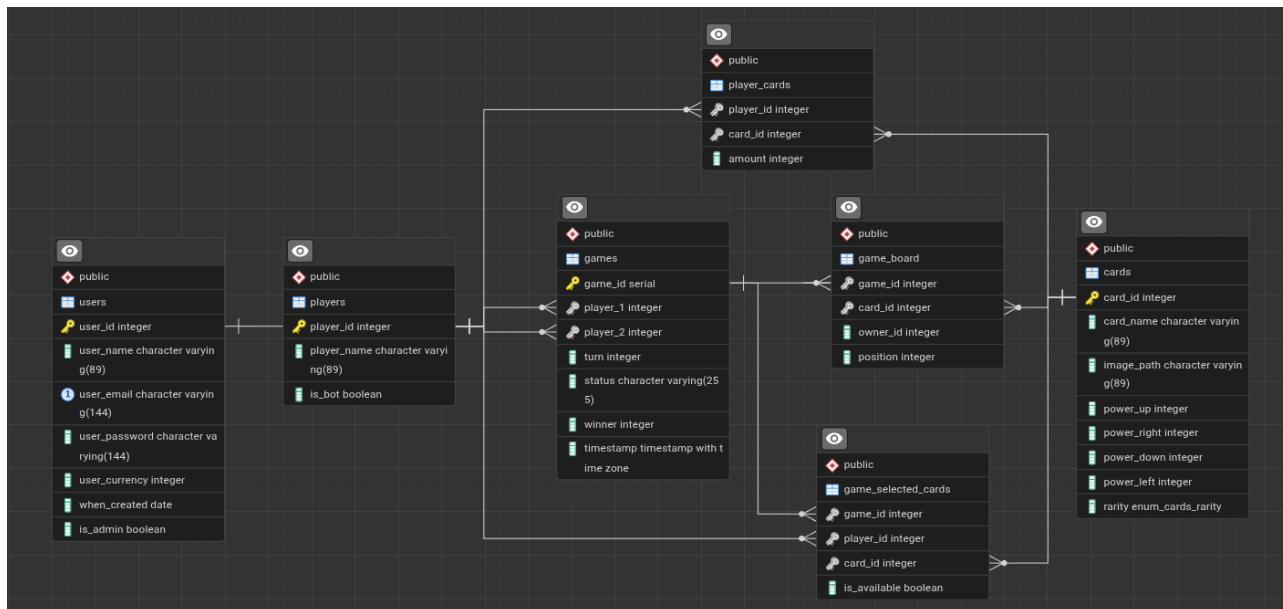
src/middleware incluye los middlewares de autenticación, autorización y tratamiento de errores;

src/utils almacena utilidades compartidas, como la clase de error personalizada;

y **public** contiene los recursos estáticos como estilos e imágenes.

La aplicación se inicia desde `src/index.js`, que configura Express, el parser de formularios y JSON, el middleware de sesiones, las vistas y el servidor estático, además de conectar la aplicación con el router principal. También se encarga de verificar la base de datos, sincronizar los modelos y ejecutar el proceso de seed si es necesario. Esta estructura de arranque centralizada es adecuada para una aplicación backend modular y facilita el control de todo el ciclo inicial del servidor.

La estructura del esquema en postgres es el siguiente:



5. Sistema de autenticación y sesiones

El sistema incorpora registro, inicio de sesión y cierre de sesión. Durante el registro, se crea un nuevo usuario asociado a un jugador, se almacena la contraseña cifrada y se inicializa la colección de cartas básica del usuario (*'hardcoded'* para cada nuevo usuario con cartas de nivel bajo).

En el inicio de sesión, se validan las credenciales y, si son correctas, se genera una sesión activa que conserva el identificador del usuario y su rol de administrador. El cierre de sesión destruye la sesión y elimina la cookie asociada.

La autenticación se apoya en *express-session*, lo que permite mantener la información del usuario en el servidor mientras navega por la aplicación. Gracias a este mecanismo, las rutas privadas pueden comprobar si existe una sesión activa antes de permitir el acceso. Además, se incluye un middleware de administrador que restringe el acceso al panel de administración únicamente a usuarios con permisos adecuados. Este sistema de control de acceso es suficiente para una aplicación de este tipo y resulta sencillo de mantener.

6. Modelo de datos

El modelo de datos está diseñado para representar las entidades principales del juego: usuarios, jugadores, cartas, colecciones, partidas, tablero y cartas seleccionadas. Cada usuario se asocia a un jugador, y cada jugador dispone de una colección con cantidades disponibles de cada carta. Las partidas almacenan los participantes, el estado actual, el turno, el ganador y otros datos relevantes

del flujo de juego. El tablero se modela como una cuadrícula de nueve posiciones, y las cartas seleccionadas en cada partida se guardan en una tabla intermedia para poder reconstruir el estado de juego.

Sequelize se utiliza como ORM para interactuar con la base de datos de forma estructurada. Esto permite trabajar con asociaciones entre modelos, realizar consultas filtradas y ejecutar operaciones de inserción, actualización y borrado de manera más clara que con SQL manual. En las operaciones más delicadas se emplean transacciones, especialmente cuando se modifican varias tablas al mismo tiempo. De esta forma se evita que la colección, la selección de cartas y las partidas queden en un estado incoherente ante un fallo parcial.

7. Rutas principales

La aplicación se organiza en rutas separadas por funcionalidad. Las rutas de autenticación permiten mostrar formularios de login y registro, procesar credenciales y cerrar sesión. Las rutas de vistas se encargan de renderizar páginas como la colección, la nueva partida y la pantalla de juego. Las rutas de administración permiten gestionar bots y asignar cartas manualmente. Por otro lado, la API expone endpoints específicos para la lógica del juego, como selección de cartas, creación de partidas, consulta del tablero, consulta del estado de la partida, movimiento de cartas y robo final.

Las rutas de autenticación son:

- GET /login
- GET /register
- POST /auth/login
- POST /auth/register
- POST /auth/logout

Las rutas de vistas principales son:

- GET /players/:id/newgame
- GET /players/:id/game/:gameId
- GET /players/:id/game/:gameId/cancel
- GET /players/:id/collection
- POST /players/:id/game-selection/add/:cardId
- POST /players/:id/game-selection/remove/:cardId
- POST /players/:id/commitgame

Las rutas de administración son:

- GET /admin
- GET /admin/assign-cards/:playerId
- POST /admin/assign-cards/
- POST /admin/createbot

- POST /admin/deletebot/:playerId

La API del juego incluye:

- GET /api/:id/collection
- GET /api/:id/collection/available
- GET /api/:id/game-selection
- POST /api/:id/game-selection/add
- POST /api/:id/game-selection/remove
- POST /api/:id/game-selection/reset
- POST /api/:id/game
- POST /api/:id/game/cancel
- GET /api/:id/positions
- POST /api/:id/:gameId/move
- GET /api/:gameId/status
- GET /api/:id/:gameId/status
- GET /api/:id/:gameId/status/full-state
- GET /api/:id/:gameId/steal
- POST /api/:id/:gameId/steal

8. Lógica funcional del juego

La partida comienza desde la pantalla de nueva partida, donde el jugador elige un bot y selecciona exactamente cinco cartas de su colección. Cuando se confirma la selección, se genera una partida y se reservan las cartas del jugador y del bot. A continuación, se inicializa el tablero con nueve casillas vacías y la partida queda lista para jugar.

Durante la partida, cada turno permite colocar una carta en una posición vacía del tablero. Cuando se sitúa una carta, el sistema comprueba automáticamente las cartas adyacentes. Si el valor de poder de la carta colocada en un lado supera al valor correspondiente de una carta rival adyacente, esa carta cambia de propietario. Esta mecánica es la base estratégica del juego, ya que obliga a pensar no sólo en la posición de la carta sino también en sus valores laterales.

Cuando el tablero se llena, el sistema cuenta cuántas cartas controla cada jugador. El jugador con mayor número de cartas capturadas pasa a la fase de robo, en la que obtiene una carta de la colección del rival. En las partidas contra bot, esta elección se automatiza escogiendo la carta más potente disponible. Si el bot participa como ganador, la selección del robo se resuelve por lógica interna sin intervención del usuario.

9. Vistas y experiencia de usuario

La interfaz se renderiza con EJS usando una plantilla general que incluye cabecera, navegación y contenido principal. La vista base centraliza la estructura común de la aplicación, mientras que cada pantalla específica se inyecta como contenido dinámico. Esto permite mantener una apariencia homogénea sin repetir código en cada página.

La barra de navegación cambia según la sesión del usuario. Si el usuario está autenticado, aparecen enlaces a la nueva partida, la colección y el cierre de sesión. Si además tiene permisos de administrador, se muestra el acceso al panel de administración. Las páginas principales incluyen la pantalla de inicio, la de registro, la de login, la colección de cartas, la nueva partida, el tablero de juego y las vistas administrativas para asignar cartas y gestionar bots. Esta estructura resulta clara para el usuario y refleja adecuadamente el estado de la sesión.

10. Middleware y tratamiento de errores

El middleware de autenticación comprueba que exista una sesión activa antes de permitir el acceso a determinadas rutas. Si no hay usuario autenticado, la petición se redirige al login. El middleware de administración amplía esta comprobación y valida que el usuario tenga permisos de administrador antes de mostrar el panel correspondiente. De este modo se protege la funcionalidad sensible de la aplicación.

El proyecto también incluye un middleware de errores centralizado. Cuando se produce una excepción, este middleware determina el código de estado y el mensaje correspondiente. Si la petición pertenece a la API, devuelve una respuesta JSON; si se trata de una ruta de vistas, renderiza una página de error. Además, se utiliza un manejador asíncrono para capturar de forma uniforme las excepciones de las funciones *async*, evitando duplicar bloques *try/catch* en cada controlador.

11. Retos técnicos y decisiones de implementación

Uno de los principales retos técnicos del proyecto ha sido mantener la consistencia de la colección de cartas del jugador mientras interactúa con la selección de mazo y la cancelación de partidas. Para ello, las operaciones se encapsulan en transacciones, de manera que si una parte del proceso falla, la base de datos no quede en un estado intermedio. También ha sido importante controlar correctamente el estado de la partida, ya que una misma sesión puede pasar por los estados de selección, activo, espera de robo y finalizada.

Otro reto ha sido la implementación del bot. El sistema necesita elegir una carta y una posición de forma coherente con el estado del tablero, intentando maximizar capturas cuando sea posible. Para ello, la lógica del bot analiza las posiciones disponibles, evalúa los poderes de las cartas y prioriza jugadas más ventajosas. Este componente dota al juego de una experiencia más dinámica y reduce la intervención manual en las partidas automáticas.

Citando otro reto, el modelo ha sido cambiado a lo largo del desarrollo para adaptar ciertas circunstancias. Como por ejemplo los 'id' generados por el ORM resultaron ser necesarios para ciertas transacciones, en un caso para quitar un sólo record de un tablero de selección en el que existen varias cartas que comparten el mismo `card_id`. El siguiente conflicto surge en la necesidad de mimetizar la PK `players.player_id` hacia la PK de `users.user_id` para gestionar la creación

incremental de los usuarios y bots. Recordemos que los bots no generan usuario, tan sólo gestionan una colección de cartas.'

La cantidad de cartas que posee un bot no es verificada ya que queremos randomizar las posibilidades de juego únicas pero dados unos valores límite y así poder graduar la dificultad. Eso obliga a hacer una selección no basada en el 'amount', sino agregar una lógica de selección de cartas enemigas que deberían ajustarse para un futuro juego multijugador.

El CRUD no se hace directamente a través de PUT, PATCH o DELETE, dado que sequelize en apariencia ya posee los métodos para hacer estas transacciones sin hacer una *raw.query* (como por ejemplo update o destroy importando el modelo), por lo que cara a futuro se podrían centralizar más controladores, servicios y modelos sin mezclar responsabilidades. Hasta se consideró a lo largo del proyecto quitar algunas relaciones ya que esto solo serviría en conceptos de administración directa de base de datos.

Existe un comportamiento no esperado en el archivo *new-game.ejs* en la que el 'submit' del form ejecuta 'commitgame' en lugar de retirar las cartas de la selección de cartas, aunque por circunstancias de tiempo no se consiguió depurar.

12. Conclusión

Quadruple Square es un proyecto backend que combina autenticación, persistencia de datos, lógica de juego y renderizado de vistas en una arquitectura mantenible. La aplicación permite gestionar usuarios, colecciones, partidas contra bots, tablero de juego y recompensas finales, todo ello mediante una estructura modular basada en MVC. El uso de Sequelize, Express y EJS permite construir una solución clara y funcional, adecuada para backend, control de sesiones, diseño de rutas y lógica transaccional.

Como posibles mejoras futuras, se podría ampliar el sistema con nuevos modos de juego, un sistema de intercambio de cartas por puntos, más tipos de bots con distintos niveles de dificultad preconfigurados, un historial de partidas por usuario (columna 'position_id' en el game_board) o incluso una interfaz frontend más avanzada que consuma la API de forma más interactiva para la funcionalidad de drag and drop. También sería interesante refinar la lógica de validación y ampliar la cobertura de errores para reforzar todavía más la robustez del sistema y quizás agregar un historial de perfil (partidas ganadas, eliminar cuenta, modificar nombre de jugador).

Cabe recalcar que en el proceso de desarrollo y por falta de tiempo se recurrió más de lo habitual a las LLM (en especial para frontend, lógica de bots y autenticación) que a pesar de a veces agrupar ideas reutilizables, tienden a generar más métodos que, o hacen conflicto con métodos existentes o se vuelven rápidamente obsoletos o generan más errores. No contemplan el estilo de código existente para imports/exports o excepciones, y termina habiendo excesivas verificaciones que no siguen ningún estandar lógico o terminan rompiendo partes del código. Como por ejemplo, para proyectar estructura de autenticación no escatima en sugerir instalar *JWT* y *express-sessions* en paralelo sin ninguna justificación. Pese a todo, la experiencia y el aprendizaje en conceptos de cifrado y seguridad cliente-servidor ha sido enriquecedor y entretenido.